Design Pattern Idempotent Receiver

Intento e definizione

L'intento del pattern è identificare univocamente le richieste dei client in modo da poter ignorare le richieste duplicate quando un client tenta nuovamente l'operazione. L'obiettivo è garantire che un'operazione eseguita più volte abbia lo stesso effetto di essere eseguita una sola volta.

Il **problema** è che, in un sistema distribuito, la comunicazione non sempre è affidabile. Un client potrebbe inviare una richiesta e non ricevere mai la risposta (perché persa o perché il server si è bloccato prima di rispondere). Il client, non sapendo se il server ha elaborato la richiesta, è costretto quindi ad inviarla di nuovo.

- **At-least-once delivery**: molti sistemi di messaggistica garantiscono che un messaggio arrivi almeno una volta per assicurare l'affidabilità. Di conseguenza, la ricezione di duplicati è un evento inevitabile che il sistema deve gestire.
- **Esempio critico**: immaginiamo un servizio di pagamento. Se si invia la richiesta "Addebita 100 euro a Bob" e, a causa di un errore di rete, viene inviata due volte, senza idempotenza Bob pagherebbe 200 euro.

Il Receiver (cioè il server) deve essere progettato per gestire queste duplicazioni in modo trasparente. La soluzione prevede i seguenti passi:

1. **Identificazione univoca**: il client assegna un ID univoco a ogni richiesta (tramite una numerazione progressiva o un hash dei dati).
2. **Memorizzazione**: il server mantiene una "sessione" o un database (MessageDB) in cui memorizza gli ID delle richieste già elaborate e le relative risposte.
3. **Verifica all'arrivo**: quando arriva una richiesta, il server controlla se l'ID è già presente nel suo storico (MessageReceiver):
    - **se esiste**: significa che è un duplicato. Il server non rielabora la richiesta, ma restituisce immediatamente la risposta che aveva memorizzato in precedenza.
    - **se non esiste**: il server elabora la richiesta, salva il risultato e l'ID, e invia la risposta.

Gestione delle risorse

Conservare la storia di tutte le richieste può occupare molta memoria. Il pattern suggerisce delle strategie di "pulizia":

- **timeout**: cancellare le richieste conservate dopo un certo periodo di tempo.
- **numerazione progressiva**: se le richieste sono numerate sequenzialmente, il server può rimuovere tutte le richieste salvate che hanno un numero inferiore all'ultima ricevuta con successo, assumendo che non verranno più ritrasmesse.

Conseguenze

L'adozione di questo pattern comporta sia benefici che costi:

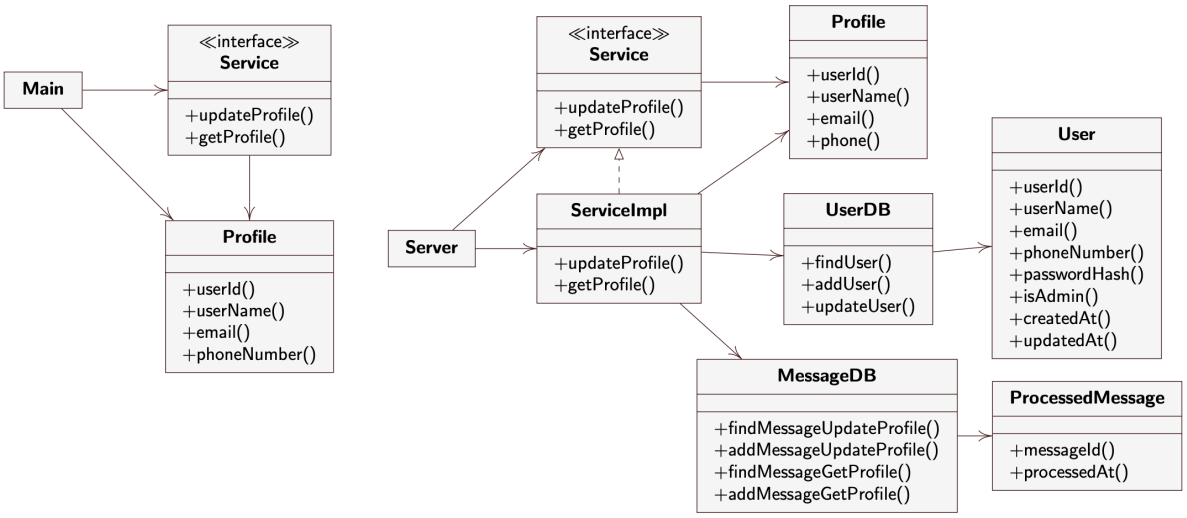
Benefici (vantaggi)

- garantisce l'effetto "**exactly-once**" (eseguito esattamente una volta) anche su infrastrutture che supportano solo "at-least-once".

- migliora la consistenza e l'affidabilità dei dati (evita doppi addebiti o doppie scritture).
- utile nelle architetture a microservizi per rendere i servizi robusti.

Compromessi (svantaggi)

- aumenta l'occupazione di memoria o storage per salvare lo storico dei messaggi.
- può aumentare i tempi di esecuzione a causa della necessità di cercare l'ID delle richieste nel database prima di ogni operazione.
- richiede una gestione complessa della scadenza delle richieste registrate (messaggi salvati).

Esempio

Service consente di leggere e aggiornare i dati di un utente. Per far si che le operazioni siano idempotenti, i messaggi sono numerati e ServiceImpl dovrà conservare i messaggi ricevuti: